Probeklausur: Mikrocomputertechnik 1

Bearbeitungszeit: 60 Minuten · Gesamtpunktzahl: 60 Punkte

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1 enthält zehn Fragen im Multiple-Choice-Format. Bei jeder Frage können eine, mehrere oder alle Antwortmöglichkeiten richtig sein. Jede vollständig korrekte Auswahl erhält einen Punkt. Die Aufgaben 2–7 sind Papieraufgaben mit den jeweils angegebenen Punkten; dort sind Teilpunkte möglich.

Name:

_____

Matrikelnummer:

_____

Kurs / Studiengruppe:

_____

Hilfsmittel: Das offizielle RISC-V Green Card / Reference Sheet wird ausgeteilt. Alle Aufgaben sind auf Papier ohne Simulator oder Compiler lösbar.

Stoffumfang: Vorlesungen 1–7 und deren Wiederholung in Einheit 9.

Markieren Sie in Aufgabe 1 alle richtigen und keine falschen Optionen. Jede andere Auswahl erhält null Punkte; es gibt keinen Punktabzug. Geben Sie bei den Aufgaben 2–7 Rechenwege beziehungsweise Begründungen an.

Aufgabe	Inhalt	Punkte	Richtzeit	erreicht
1	10 Multiple-Choice-Fragen, Vorlesungen 1–7	10	10 min	
2	Zahlendarstellung und Logik	8	8 min	
3	Array und Schleife in RV32I	10	10 min	
4	Funktionsaufruf und Stacktrace	8	8 min	
5	Datenpfad und Steuerwerk	8	8 min	
6	Pipeline und Laufzeit	10	10 min	
7	Maschinencode und Bytefolge	6	6 min	
	Summe	**60**	**60 min**	

Die Richtzeiten umfassen Lesen und Bearbeiten und dienen der Zeiteinteilung.

Gemeinsame Annahmen: RV32I ohne komprimierte Instruktionen, 32-Bit-Register, Little Endian und Byteadressierung. Echte Instruktionen sind vier Byte lang. Registerarithmetik behält die unteren 32 Bit. Normale Funktionsaufrufe verwenden die ILP32-Konvention: skalare Ergebnisse in `a0`, Stackzeiger während standardkonformer Funktionen auf 16 Byte ausgerichtet. Die Arrayelemente sind 32-Bit-Wörter. Datenadressen und Speicherbereiche sind gültig, soweit nicht ausdrücklich nach einer Bereichsgrenze gefragt wird.

Datenpfad und Pipeline sind die jeweils ausdrücklich angegebenen Lehrmodelle. Zeit- und Adresswerte sind Aufgabenannahmen, keine gemessenen Eigenschaften eines Prozessors oder Simulators.

Aufgabe 1: Multiple Choice (10 Punkte)

Fragen 1–4

Frage 1: ISA und Umsetzung (1 Punkt)

Zwei Prozessoren führen denselben RV32I-Code korrekt aus, benötigen dafür aber verschieden viele Takte. Welche Aussagen treffen zu?

- ☐ A: Beide können dieselbe ISA mit unterschiedlichen Mikroarchitekturen implementieren.
- ☐ B: Die unterschiedliche Taktzahl verlangt unterschiedliche RV32I-Instruktionscodierungen.
- ☐ C: Die Laufzeit lässt sich allein aus den Assemblerzeilen bestimmen.
- ☐ D: Die ISA beschreibt das sichtbare Verhalten, nicht jede interne Taktfolge.

Frage 2: Bitmuster und Zahlenwert (1 Punkt)

Das achtbittige Muster `1111 1011` wird ohne Änderung der Bits betrachtet. Welche Aussagen treffen zu?

- ☐ A: Als vorzeichenloser Wert stellt es die Zahl 251 dar.
- ☐ B: Als Zweierkomplementwert stellt es die Zahl −5 dar.
- ☐ C: Mit Vorzeichenerweiterung wird daraus `0xFFFFFFFB`.
- ☐ D: Mit Nullerweiterung wird daraus `0x000000FB`.

Frage 3: Logik und gespeicherter Zustand (1 Punkt)

Ein Multiplexer und ein positiv flankengetriggertes Register werden ohne Reset betrachtet. Welche Aussagen treffen zu?

- ☐ A: Ein Multiplexer kann seinen Ausgang nach einer Auswahländerung zwischen Flanken ändern.
- ☐ B: Ein Multiplexer übernimmt seine neue Auswahl erst an der nächsten Taktflanke.
- ☐ C: Das Register folgt bei dauerhaft hohem Takt jeder Änderung seines Dateneingangs.
- ☐ D: Ohne neue aktive Flanke bleibt der gespeicherte Registerwert erhalten.

Frage 4: Register und Immediates (1 Punkt)

Für RV32I ohne Erweiterungen: Welche Aussagen treffen zu?

- ☐ A: Nach `addi x0, x0, 7` enthält das Register `x0` den Wert sieben.
- ☐ B: Das Immediate −2048 ist bei `addi` direkt codierbar.
- ☐ C: Die Anweisung `addi t0, t1, 2048` verwendet ein zulässiges Immediate.
- ☐ D: Eine Addition über die 32-Bit-Grenze hinaus löst einen Überlauf-Trap aus.

Fragen 5–8

Frage 5: Speicher und Erweiterung (1 Punkt)

Das Wort `0x00000080` liegt ab Adresse `0x2000` in Little Endian. Die Zugriffe sind gültig. Welche Aussagen treffen zu?

- ☐ A: An Adresse `0x2000` liegt das Byte `0x00`.
- ☐ B: `lb t0, 0(a0)` mit `a0=0x2000` liefert `0xFFFFFF80`.
- ☐ C: `lbu t0, 0(a0)` mit `a0=0x2000` liefert `0x00000080`.
- ☐ D: Das Basisregister `a0` wird durch den Load um ein Byte erhöht.

Frage 6: Bedingte Sprünge (1 Punkt)

Vor dem Branch gilt `t0=0xFFFFFFFF` und `t1=1`. Welche Aussagen treffen zu?

- ☐ A: `blt t0, t1, ziel` nimmt den Branch.
- ☐ B: `bltu t0, t1, ziel` nimmt den Branch.
- ☐ C: `bgeu t0, t1, ziel` nimmt den Branch.
- ☐ D: `beq t0, t1, ziel` nimmt den Branch.

Frage 7: Aufrufkonvention (1 Punkt)

Eine normal aufgerufene ILP32-Funktion verändert `s0` und ruft mit `jal ra, helfer` eine weitere Funktion auf. Welche Aussagen treffen zu?

- ☐ A: Der Helfer muss alle vom Aufrufer noch benötigten `t`-Register erhalten.
- ☐ B: Sie muss ihren eingehenden `s0`-Wert vor der eigenen Rückkehr wiederherstellen.
- ☐ C: Sie muss ihre äußere Rückkehradresse über den Helferaufruf hinweg erhalten.
- ☐ D: Das Pseudokommando `ret` stellt den alten Stackpointer automatisch wieder her.

Frage 8: Hauptsteuerung und ALU-Steuerung (1 Punkt)

Im Kursmodell werden `add` und `sub` mit gleichem Opcode, aber unterschiedlichen gültigen Funktionsfeldern ausgeführt. Welche Aussagen treffen zu?

- ☐ A: Die beiden Befehle benötigen unterschiedliche Werte des Signals `RegWrite`.
- ☐ B: Der siebenbittige Opcode unterscheidet bereits beide Rechenoperationen.
- ☐ C: Die vier Bits von `ALUControl` sind ein vorgeschriebenes RISC-V-Instruktionsfeld.
- ☐ D: Die ALU-Detailsteuerung unterscheidet sie anhand der Funktionsfelder.

Fragen 9–10

Frage 9: Laufzeitvergleich (1 Punkt)

Zwei Ausführungen haben jeweils `N=100` Maschineninstruktionen. A hat `CPI=1` und eine Taktperiode von `4 ns`, B hat `CPI=2` und `1 ns`. Welche Aussagen treffen zu?

- ☐ A: Die Ausführung A benötigt insgesamt eine Zeit von 400 ns.
- ☐ B: Die Ausführung A ist wegen ihres kleineren CPI schneller.
- ☐ C: Die Ausführung B benötigt insgesamt eine Zeit von 200 ns.
- ☐ D: Der Quotient Zeit A durch Zeit B beträgt zwei.

Frage 10: Forwarding (1 Punkt)

Die Fünfstufen-Kurspipeline führt mit Forwarding `add t0, t1, t2`, `addi t0, t0, 4` und `sub a0, t0, t4` aus. Anfangs gelten `t1=6`, `t2=5` und `t4=3`. Welche Aussagen treffen zu?

- ☐ A: Der Verbraucher `sub` muss das ältere Ergebnis elf verwenden.
- ☐ B: Für `sub` hat der Treffer aus EX/MEM Vorrang vor MEM/WB.
- ☐ C: Der letzte Befehl schreibt den Wert zwölf nach `a0`.
- ☐ D: `add` und `addi` erzeugen verschiedene Versionen von `t0`.

Aufgabe 2: Zahlendarstellung und Logik (8 Punkte)

a) Stellen Sie −37 als achtbittigen Zweierkomplementwert in Binär- und Hexadezimalschreibweise dar. Zeigen Sie den Rechenweg ausgehend von +37. (3 Punkte)

b) Addieren Sie die achtbittigen Muster `0111 1111` und `0000 0001`. Geben Sie Ergebnis und hinausfallenden Übertrag an. Liegt bei signed Interpretation ein Überlauf vor? Begründen Sie. (2 Punkte)

c) Ein Einbit-Multiplexer wählt bei $S = 0$ den Eingang A, bei $S = 1$ den Eingang B. Geben Sie die Boolesche Gleichung für Y an und füllen Sie die vier Tabellenzeilen aus. (3 Punkte)

S	A	B	Y
0	1	0	
1	1	0	
1	0	1	
0	0	1	

Rechenwege und Gleichung:

Aufgabe 3: Array und Schleife in RV32I (10 Punkte)

Die Blattfunktion `sum_first` soll die ersten `n` Elemente eines Arrays addieren und die Summe zurückgeben. Für `n=0` liefert sie null, ohne ein Arrayelement zu lesen.

Es gilt $0 \leq n \leq 4$. Ab `0x3000` liegen vier 32-Bit-Zweierkomplementwerte lückenlos: −2, 5, 7, −3. Bei Eintritt enthält `a0` die Basisadresse `0x3000`, `a1` die vorzeichenlose Länge `n`. Ein Element mit Index `i` steht vier mal `i` Byte hinter der Basis; die Indizes beginnen bei null. Die Funktion benötigt keine Unteraufrufe und darf ohne Stackframe auskommen.

a) Ergänzen Sie die sechs mit _____ markierten Lücken. `t0` hält die unveränderte Arraybasis, `t1` den Index und `t2` die Summe. Die Abbruchbedingung verwendet einen unsigned Vergleich; die Rückgabe gehört in `a0`. (6 Punkte)

```
sum_first:
    addi t0, a0, 0
    addi t1, x0, 0
    addi t2, x0, 0
sum_loop:
    ____ t1, a1, sum_end
    slli t3, t1, _____
    add  t4, _____, t3
    lw   t5, _____(t4)
    add  t2, t2, _____
    addi t1, t1, 1
    jal  x0, sum_loop
sum_end:
    addi a0, _____, 0
    jalr x0, 0(ra)
```

b) Bestimmen Sie für `n=0` und `n=3` jeweils den Rückgabewert und die Zahl der Array-Loads. (2 Punkte)

c) Geben Sie die Byteadresse und den gespeicherten Zahlenwert des Elements mit Index zwei an. Begründen Sie die Skalierung des Index in der Adressrechnung. (2 Punkte)

Ergebnisse und Begründung:

Aufgabe 4: Funktionsaufruf und Stacktrace (8 Punkte)

Vor `PC=0x1000` gilt `a0=4`, `s0=99`, `sp=0x2000`. Der Stackbereich ist beschreibbar. Alle Zeilen sind echte Vier-Byte-Instruktionen; die angegebenen Adressen und Labels gelten für diesen Papierausschnitt.

```
0x1000: jal ra, F
0x1004: addi t0, a0, 0
F:
0x1100: addi sp, sp, -16
0x1104: sw ra, 12(sp)
0x1108: sw s0, 8(sp)
0x110C: addi s0, a0, 0
0x1110: jal ra, Double
0x1114: add a0, a0, s0
0x1118: lw s0, 8(sp)
0x111C: lw ra, 12(sp)
0x1120: addi sp, sp, 16
0x1124: jalr x0, 0(ra)
Double:
0x1200: slli a0, a0, 1
0x1204: jalr x0, 0(ra)
```

a) Tragen Sie den Zustand unmittelbar **nach** der jeweiligen Instruktion ein. `PC danach` bezeichnet die als Nächstes auszuführende Adresse. Geben Sie `a0` und `s0` dezimal, die übrigen Größen hexadezimal an. (5 Punkte)

gerade ausgeführt	PC danach	a0	s0	ra	sp
0x1110					
0x1124					

b) Geben Sie für beide Stores die effektive Adresse und das dort abgelegte 32-Bit-Wort in Hexadezimalschreibweise an. (2 Punkte)

c) Nur die Instruktion `lw ra, 12(sp)` wird durch eine wirkungslose Instruktion ersetzt; alle angegebenen Adressen bleiben unverändert. Welche Adresse wählt dann der erste Sprung bei `0x1124`? (1 Punkt)

Adressen, Wörter und Begründungen:

Aufgabe 5: Datenpfad und Steuerwerk (8 Punkte)

Verwenden Sie den Single-Cycle-Kursausschnitt für `add`, `sub`, `and`, `or`, `lw`, `sw` und `beq`. `ALUSrc` wählt 0 für die zweite Registerquelle oder 1 für das Immediate; `MemtoReg` wählt 0 für das ALU-Ergebnis oder 1 für Speicherlesedaten. Register- und Speicherschreiben haben getrennte Freigaben.

ImmGen liefert bei Branches den fertigen vorzeichenerweiterten Byteoffset; danach erfolgt kein weiterer Shift. Die ALU subtrahiert die Registerwerte zum Gleichheitstest.

a) Füllen Sie die Hauptsteuerzeilen mit 0, 1 oder X (Don't Care). Bei einer unerheblichen Auswahl sind auch begründete konkrete Werte zulässig. (6 Punkte)

Befehl	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`lw`						
`beq`						

b) Ein `beq` steht bei `PC=0x4000`. Beide Quellen enthalten 13; ImmGen liefert −12. Bestimmen Sie `Zero`, `PCSrc` und den nächsten PC. (2 Punkte)

Rechenwege und Begründung:

Aufgabe 6: Pipeline und Laufzeit (10 Punkte)

Fünf Stufen: IF (Befehl holen), ID (decodieren und Register lesen), EX (rechnen), MEM (Datenzugriff), WB (Rückschreiben / Abschluss). Anfangs leere Pipeline, höchstens ein neuer Befehl pro Takt, getrennte Befehls- und Datenzugriffe; jeder Load braucht eine MEM-Stufe. Forwarding aus EX/MEM und MEM/WB und Load-Use-Erkennung sind vorhanden. Der jüngste passende Produzent ist maßgeblich; ist sein Wert noch nicht bereit, muss gewartet werden. Ein Schreibversuch auf `x0` erzeugt keine Forwardingquelle. Ein Load liefert seine Daten erst nach MEM. Der Stall hält PC und IF/ID und erzeugt eine Bubble in ID/EX; ältere Arbeit läuft weiter.

Vor dem Ausschnitt steht `s0=0x3000`; dort liegt das Wort 7:

```
lw   t0, 0(s0)
addi t0, t0, 5
add  a0, t0, t0
```

a) Tragen Sie die Stufenfolge ein und kennzeichnen Sie den Stall. Wiederholtes ID beziehungsweise IF mit Stall-Markierung ist zulässig. (5 Punkte)

Befehl / Takt	1	2	3	4	5	6	7	8
`lw t0, 0(s0)`								
`addi t0, t0, 5`								
`add a0, t0, t0`								

b) Begründen Sie den Stall und erklären Sie, warum zwischen `addi` und `add` kein weiterer Stall nötig ist. (2 Punkte)

c) Ein **anderer** anfangs leerer Pipeline-Lauf umfasst 100 gültig abgeschlossene Maschineninstruktionen, acht Load-Use-Stallzyklen und drei genommene Branches. Die Branch-Auflösung erfolgt in EX; jeder genommene Branch kostet zwei zusätzliche Takte. Die Zusatzkosten überschneiden sich nicht, weitere Stalls gibt es nicht. Die Taktperiode beträgt 2 ns. Bestimmen Sie Gesamttakte, Gesamtzeit und Speedup gegenüber einer Ausführung derselben 100 Instruktionen mit CPI eins und 8 ns Taktperiode. Gezählt wird vom ersten IF bis Ende des letzten WB; Bubbles und verworfene Einträge zählen nicht zu den 100 Instruktionen. (3 Punkte)

Begründungen und Rechnungen:

Aufgabe 7: Store codieren und im Speicher darstellen (6 Punkte)

Ein Array aus sechs 32-Bit-Wörtern beginnt bei `0x6000`; die Indizes beginnen bei null. `s1` (`x9`) zeigt auf das Element mit Index fünf. `a2` (`x12`) enthält den Wert 13, der das Element mit Index zwei überschreiben soll. Alle Speicherzugriffe sind gültig und ausgerichtet. Verwenden Sie `s1` als Basisregister.

a) Bestimmen Sie die Zieladresse und den vorzeichenbehafteten Byteoffset relativ zu `s1`. (1 Punkt)

b) Geben Sie eine einzelne native RV32I-Instruktion für diesen Speicherzugriff an. Codieren Sie sie: Benennen Sie die Felder des verwendeten Formats, tragen Sie deren Inhalte binär ein und bilden Sie das vollständige 32-Bit-Maschinenwort in Hexadezimalschreibweise. Nutzen Sie das ausgeteilte Referenzblatt. (4 Punkte)

c) Das Instruktionswort aus b) liegt ab Byteadresse `0x1800` im Little-Endian-Speicher. Geben Sie die vier Bytewerte mit ihren Adressen an. (1 Punkt)

Adressrechnung, Instruktion, Felder und Bytefolge: